Transformer (IceCube 2D) Summary(1FA006)

Dabbugunta Hyndavi Anand

The Jupyter notebook and the plots can be found in this GitHub repository.

The aim of this assignment was to implement a Transformer to predict neutrino event positions $(x_{\text{pos}}, y_{\text{pos}})$ from hit data. The dataset consisted of jagged arrays stored in Parquet format. The units of the position co-ordinates is assumed to be metres (m).

1 Methodology

Train, validation, and test sets were normalized and then loaded with batch size (B) 128. Each event contained hit-level features (t, x, y) with shape $(3, N_{\text{hits}})$ and event-level labels $(x_{\text{pos}}, y_{\text{pos}})$.The given collate function concatenates the event hits to input into the encoder.

The forward pass of `TransformerEncoder` instance looks like follows :

1. Input hits: $(B \times N,\ 3)$ — time, x, y
2. Input embedding: Linear$(3 \to 64)$:
$$(B \times N,\ 3) \xrightarrow{\text{Linear}} (B \times N,\ 64)$$
3. Split, pad, positional embedding:
$$(B \times N,\ 64) \to (B,\ L_{\max},\ 64)$$
4. Transformer encoder: (2 layers, 4 heads):
$$(B,\ L_{\max},\ 64) \to (B,\ L_{\max},\ 64)$$
5. Attention pooling:
$$(B,\ L_{\max},\ 64) \to (B,\ 64)$$
6. MLP output head $(64 \to 32 \to 2)$:
$$(B,\ 64) \to (B,\ 2)$$

Training used Adam with learning rate 10^{-4}, cosine LR schedule, and early stopping (patience 15). Losses were logged to CSV.

2 Results

The results of the model were satisfactory as described in the plots file. At epoch 82, the train loss was around 0.065 and the validation loss was around 0.068. The average test loss was about 0.064.

3 Challenges encountered

With the given template and some blocks of code from the GNN assignment, I could get everything up and running. At first, my loss curve showed overfitting from early on (around epoch 25) and it was the problem even after trying other hyperparameters. Then I realized that I forgot to add the positional embedding into the padded inputs. After adding the positional embedding the performance was better but still the Mean Absolute Error (MAE) was almost 0.9m and I wanted to improve that. So, ChatGPT suggested I make an MLP to assign a score or a weight to every hit when pooling. This made sense because some hits could be noise and giving every hit equal weight could adversely affect the accuracy. After changing the mean pooling to attention pooling, the MAE improved to 0.68m (x_{pos}) and 0.82m (y_{pos}).